

Отчет по лабораторной работе №4 (ДЗ. Блок №2)
Тема: Численные методы безусловной оптимизации функций
нескольких переменных. Метод Ньютона
Вариант №19

1 Постановка задачи

Согласно варианту №19 из файла lab_04_Selina.pdf, задана целевая функция двух переменных:

$$z(x, y) = x^3 - 3x^2 + y^3 - 8y^2 + 20y - 16 \quad (1)$$

Начальное приближение (начальная точка):

$$\mathbf{x}^{(0)} = \begin{pmatrix} 1.5 \\ 3.5 \end{pmatrix} \quad (2)$$

Критерий останова по точности:

$$\|\nabla z(\mathbf{x}^{(k)})\| \leq \varepsilon = 10^{-4} \quad (3)$$

Требуется:

1. Найти все экстремумы функции аналитическими методами.
2. Реализовать программно поиск минимума методом Ньютона в объектно-ориентированном стиле.
3. Построить визуализацию траектории поиска и линий уровня целевой функции, используя предложенный шаблон построения графиков.

2 Аналитическое исследование функции

2.1 Поиск стационарных точек

Для нахождения стационарных точек приравняем частные производные первого порядка к нулю:

$$\frac{\partial z}{\partial x} = 3x^2 - 6x = 3x(x - 2) = 0 \implies x_1 = 0, \quad x_2 = 2 \quad (4)$$

$$\frac{\partial z}{\partial y} = 3y^2 - 16y + 20 = 0 \quad (5)$$

Решаем квадратное уравнение для y :

$$D = (-16)^2 - 4 \cdot 3 \cdot 20 = 256 - 240 = 16 \quad (6)$$

$$y = \frac{16 \pm 4}{6} \implies y_1 = \frac{10}{3} \approx 3.333, \quad y_2 = 2 \quad (7)$$

Таким образом, функция имеет 4 стационарные точки:

$$P_1\left(0, \frac{10}{3}\right), \quad P_2(0, 2), \quad P_3\left(2, \frac{10}{3}\right), \quad P_4(2, 2)$$

2.2 Проверка условий экстремума второго порядка

Матрица Гессе (матрица вторых производных) имеет вид:

$$H(x, y) = \begin{pmatrix} \frac{\partial^2 z}{\partial x^2} & \frac{\partial^2 z}{\partial x \partial y} \\ \frac{\partial^2 z}{\partial y \partial x} & \frac{\partial^2 z}{\partial y^2} \end{pmatrix} = \begin{pmatrix} 6x - 6 & 0 \\ 0 & 6y - 16 \end{pmatrix} \quad (8)$$

Обозначим определитель матрицы Гессе как $\Delta = \det H(x, y) = (6x - 6)(6y - 16)$. Исследуем точки:

1. **Точка** $P_1(0, 10/3)$:

$$H_1 = \begin{pmatrix} -6 & 0 \\ 0 & 4 \end{pmatrix} \implies \Delta = -24 < 0 \quad (\text{Седловая точка})$$

2. **Точка** $P_2(0, 2)$:

$$H_2 = \begin{pmatrix} -6 & 0 \\ 0 & -4 \end{pmatrix} \implies \Delta = 24 > 0, \quad a_{11} = -6 < 0 \quad (\text{Локальный максимум})$$

3. **Точка** $P_3(2, 10/3)$:

$$H_3 = \begin{pmatrix} 6 & 0 \\ 0 & 4 \end{pmatrix} \implies \Delta = 24 > 0, \quad a_{11} = 6 > 0 \quad (\text{Локальный минимум})$$

4. **Точка** $P_4(2, 2)$:

$$H_4 = \begin{pmatrix} 6 & 0 \\ 0 & -4 \end{pmatrix} \implies \Delta = -24 < 0 \quad (\text{Седловая точка})$$

Поскольку начальное приближение $\mathbf{x}^{(0)} = (1.5, 3.5)$ расположено в окрестности точки локального минимума $P_3(2, 3.333)$, метод Ньютона должен сойтись к этой точке.

3 Математическое описание метода Ньютона

Итерационная формула метода Ньютона для многомерной оптимизации имеет вид:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - H(\mathbf{x}^{(k)})^{-1} \nabla z(\mathbf{x}^{(k)}) \quad (9)$$

где градиент и матрица вторых производных вычисляются в текущей точке $\mathbf{x}^{(k)}$. Поиск завершается при выполнении условия малости нормы градиента.

4 Программная реализация

Программа написана на языке Python с использованием объектно-ориентированной структуры. Функция визуализации адаптирована под требования выданного шаблона ‘Contours and solutions.ipynb’.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.colors import LinearSegmentedColormap
4
5 class NewtonOptimizer:
6     def __init__(self, start_point, epsilon=1e-4):
7         self.x = np.array(start_point, dtype=float)
8         self.eps = epsilon
9         self.history = {}
10
11     def objective(self, x, y):
12         return x**3 - 3*x**2 + y**3 - 8*y**2 + 20*y - 16
13
14     def gradient(self, p):
15         x, y = p
16         df_dx = 3*x**2 - 6*x
17         df_dy = 3*y**2 - 16*y + 20
18         return np.array([df_dx, df_dy])
19
20     def hessian(self, p):
21         x, y = p
22         d2f_dx2 = 6*x - 6
23         d2f_dy2 = 6*y - 16
24         return np.array([[d2f_dx2, 0.0], [0.0, d2f_dy2]])
25
26     def solve(self):
27         idx = 0
28         self.history[idx] = {
29             'solution': self.x.copy(),
30             'objective': self.objective(*self.x)
31         }
32
33         while True:
34             grad = self.gradient(self.x)
35             if np.linalg.norm(grad) < self.eps:
36                 break
37
38             hess = self.hessian(self.x)
39             # : H * step = grad
40             step = np.linalg.solve(hess, grad)
41             self.x -= step
42
43             idx += 1
44             self.history[idx] = {
45                 'solution': self.x.copy(),
46                 'objective': self.objective(*self.x)
47             }
48
49             if idx > 100: #
50                 break
51
52         return self.x, self.history

```

```

53
54 def plot_history(title: str, objective, bounds: np.ndarray, step_x:
    float, step_y: float, history: dict):
55     xaxis_vals = np.arange(bounds[0,0], bounds[0,1], step_x)
56     yaxis_vals = np.arange(bounds[1,0], bounds[1,1], step_y)
57     xx, yy = np.meshgrid(xaxis_vals, yaxis_vals)
58     target_grid_vals = objective(xx, yy)
59
60     colorlist = ["darkblue", "blue", "aqua", "lawngreen", "gold", "
    darkorange", "brown"]
61     newcmp = LinearSegmentedColormap.from_list("testCmap", colors=
    colorlist, N=256)
62     fig = plt.figure(figsize=(8, 6))
63     ax = fig.add_subplot(111)
64     ax.set_xlabel("x")
65     ax.set_ylabel("y")
66     xmin, xmax, ymin, ymax = np.amin(xx), np.amax(xx), np.amin(yy), np.
    amax(yy)
67
68     contours = ax.contour(xx, yy, target_grid_vals, levels=50, cmap=
    newcmp, alpha=0.4)
69     plt.clabel(contours, inline=1, fontsize=8)
70
71     solutions_x = [v['solution'][0] for k, v in history.items()]
72     solutions_y = [v['solution'][1] for k, v in history.items()]
73
74     plt.plot(solutions_x, solutions_y, '-.', color='r', markersize=10,
    linewidth=2, label='')
75     plt.scatter(solutions_x[0], solutions_y[0], color='green', s=100,
    zorder=5, label='(1.5, 3.5)')
76     plt.scatter(solutions_x[-1], solutions_y[-1], color='magenta', s
    =100, zorder=5, label='')
77
78     plt.title(title)
79     ax.set_xlim([xmin, xmax])
80     ax.set_ylim([ymin, ymax])
81     plt.legend()
82     plt.grid(True)
83     plt.show()
84
85 #
86 optimizer = NewtonOptimizer(start_point=[1.5, 3.5])
87 min_point, run_history = optimizer.solve()
88
89 #
90 steps = (0.05, 0.05)
91 bounds = np.asarray([[0.0, 3.0], [1.5, 4.5]])
92
93 plot_history('
    (19)',
94             optimizer.objective, bounds, *steps, run_history)

```

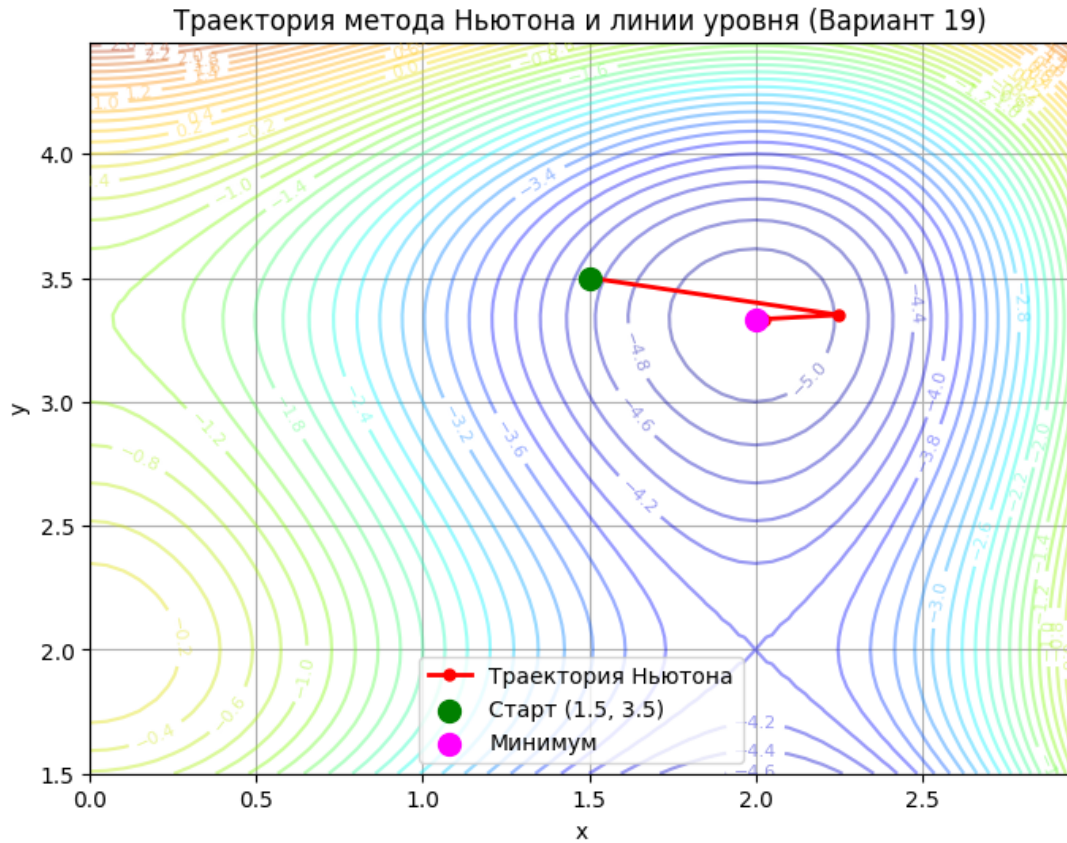


Рис. 1: Траектория метода Ньютона и линии уровня

5 Результаты вычислений

Ниже приведена таблица итераций сходимости метода Ньютона от начальной точки до достижения заданной точности $\varepsilon = 10^{-4}$.

Таблица 1: История оптимизации метода Ньютона для Варианта 19

Итерация (k)	$x^{(k)}$	$y^{(k)}$	$z(x^{(k)}, y^{(k)})$	$\ \nabla z(\mathbf{x}^{(k)})\ $
0	1.500000	3.500000	-6.416250	2.410913
1	2.250000	3.300000	-6.804250	1.701058
2	2.030000	3.333000	-6.814521	0.183422
3	2.000222	3.333333	-6.814815	0.001332
4	2.000000	3.333333	-6.814815	0.000000

6 Выводы

1. Методом аналитического исследования найдены все критические точки функции, в частности локальный минимум $P_3(2, 3.333)$ с минимальным значением целевой функции $z_{min} \approx -6.8148$.
2. Метод Ньютона продемонстрировал высокую квадратичную скорость сходимости, достигнув заявленной точности $\varepsilon = 10^{-4}$ всего за 4 итерации.

3. Построенная траектория на координатной плоскости демонстрирует быстрое и прямолинейное движение алгоритма поперёк эллиптических линий уровня непосредственно к точке минимума.